Bodo Bogdan-Mihai, Grupa 244

Tema Servlet - Wordle in limba romana

Ce face aplicatia

Am facut un joc de tip Wordle, dar cu cuvinte romanesti de 5 litere (fara diacritice, fara cratima). Utilizatorul are 6 incercari sa ghiceasca un cuvant ales aleator. Dupa fiecare incercare, fiecare litera introdusa e marcata astfel:

- verde (correct) - litera e pe pozitia buna
- galben (misplaced) - litera exista in cuvant, dar pe alta pozitie
- gri (wrong) - litera nu exista in cuvant

Jocul se joaca dintr-o pagina web facuta in React, care vorbeste cu partea de backend prin HTTP/JSON.

Arhitectura

Aplicatia e impartita in doua servlete care comunica intre ele prin HTTP, ca niste microservicii:

1. **WordsDatabase** - tine lista de cuvinte si stie sa dea un cuvant random. Rulat pe **WildFly**.
2. **GameService** - tine sesiunile de joc (cine ce cuvant are de ghicit, cate incercari a folosit) si valideaza incercarile jucatorului. Rulat pe **Tomcat**.

Frontendul vorbeste doar cu GameService. GameService, la randul lui, cere un cuvant de la WordsDatabase cand incepe un joc nou.

```
Frontend (React)  --POST /api/game-->  GameService (Tomcat)  --GET
/api/word-->  WordsDatabase (WildFly)
      |                                          |
      |<------------ sessionId ----------------|
      |                                          |
      |--POST /api/guess (sessionId, input)-->|
      |<---- rezultat litera cu litera --------|
```

Cele doua servlete + frontendul se ridica toate odata cu `docker compose up`.
Fiecare e intr-un container separat, iar WordsDatabase citeste lista de cuvinte dintr-un

volum docker partajat (`words-data`), populat la pornire de un container auxiliar care copiaza `words.txt` in volum.

De ce asa

Am ales sa separ "baza de date" de cuvinte de logica jocului pentru ca cerinta temei e sa avem doua servlete care comunica intre ele, nu doar unul mare care face tot. In plus, chiar daca aici lista de cuvinte e doar un fisier text, separarea asta seamana cu ce s-ar intampla real intr-un sistem cu microservicii: un serviciu de date si un serviciu de logica, care nu stiu nimic unul de implementarea celuilalt, doar de API-ul HTTP.

Alternativa ar fi fost sa pun totul intr-un singur razboi (WAR) cu toate servleturile, deployat o singura data. Dar atunci nu as mai fi avut doua servere de aplicatii folosite simultan, cum cere tema.

Obiecte si clase principale

WordsDatabase

WordRepository (singleton)

- `getInstance(): WordRepository` - returneaza instanta unica
- `randomWord(): String` - cuvant ales aleator din lista incarcata
- `size(): int` - cate cuvinte sunt incarcate

La prima utilizare, incarca lista de cuvinte fie din `/data/words.txt` (volumul partajat din docker), fie din resursele aplicatiei (cand ruleaza local, fara docker). Cuvintele sunt validate (exact 5 litere) si sortate alfabetic, dupa cum cerea tema.

WordServlet - mapat pe `GET /api/word`. Nu primeste parametri, raspunde cu `{"word": "CARTE"}`.

GameService

GameSession

- constructor `GameSession(String sessionId, String word)`
- `getAttemptsRemaining(): int`
- `incrementAttempts(): void`
- `getStatus(): GameStatus` / `setStatus(GameStatus status)` - starea jocului, poate fi `IN_PROGRESS`, `WON`, `LOST`

SessionStore (singleton) - tine sesiunile active intr-un `ConcurrentHashMap`

- `getInstance(): SessionStore`

- `save(GameSession session): void`
- `get(String sessionId): GameSession`

WordsDbClient - face apelul HTTP catre WordsDatabase

- `fetchWord(): String` - arunca `IOException` daca WordsDatabase nu raspunde

GuessEvaluator - compara incercarea cu cuvantul secret

- `evaluate(String word, String guess): LetterResult[]` - algoritmul clasic in doua treceri: intai se marcheaza literele exacte, apoi, pentru restul, se cauta daca litera mai exista in cuvant pe o pozitie nefolosita inca (ca sa functioneze corect cand ai litere duplicate in cuvant)

GameServlet - `POST /api/game`. Cere un cuvant nou de la WordsDatabase, creeaza o sesiune noua cu un `sessionId` random (UUID) si raspunde cu `sessionId`, lungimea cuvantului si numarul maxim de incercari. Cuvantul nu e trimis niciodata catre frontend in clar.

GuessServlet - `POST /api/guess`, body `{"sessionId": "...", "input": "AEIOU"}`. Valideaza sesiunea si lungimea cuvantului introdus, calculeaza rezultatul cu `GuessEvaluator`, incrementeaza numarul de incercari si actualizeaza starea jocului daca e cazul. Daca jocul s-a terminat (castigat sau nu mai are incercari), raspunsul contine si cuvantul corect.

CorsFilter - filtru aplicat pe toate requesturile catre GameService, ca sa poata fi apelat din browser de pe alta origine (frontendul ruleaza pe alt port).

Format raspunsuri

`POST /api/game`:

```
{ "sessionId": "...", "wordLength": 5, "maxAttempts": 6,
"attemptsRemaining": 6 }
```

`POST /api/guess`:

```
{
  "input": ["A","E","I","O","U"],
  "response": ["wrong","misplaced","wrong","correct","wrong"],
  "attemptsRemaining": 5,
  "status": "IN_PROGRESS"
}
```

Cum se ruleaza

Din radacina proiectului:

```
docker compose up --build
```

Asta porneste, in ordine: containerul care umple volumul cu `words.txt`, apoi WordsDatabase pe WildFly, apoi GameService pe Tomcat, apoi frontendul (servit cu nginx). La final:

- frontend: http://localhost:3000
- GameService: http://localhost:8081/gameservice
- WordsDatabase ramane intern, accesibil doar din reteaua docker (nu are rost sa fie expus in afara)

Cateva teste facute manual

- Start joc nou -> primesc sessionId si pot incepe sa tastez.
- Introduc un cuvant de 5 litere care nu exista deloc in cuvantul secret -> toate literele apar gri.
- Introduc chiar cuvantul corect -> toate literele apar verde, jocul se marcheaza ca WON, apare bannerul de victorie.
- Introduc un cuvant cu litere duplicate (ex. cuvantul are o singura litera A, iar eu incerc un cuvant cu doua A-uri) -> doar una din cele doua litere A e marcata misplaced/correct, cealalta iese wrong, exact cum ar trebui dupa algoritmul din Wordle original.
- Epuizez cele 6 incercari fara sa ghicesc -> jocul se marcheaza LOST si mi se arata cuvantul corect.
- Opresc containerul WordsDatabase si incerc sa pornesc un joc nou -> GameService raspunde cu eroare 503, nu pica toata aplicatia.